



UNITED INTERNATIONAL UNIVERSITY (UIU)

Dept. of Computer Science & Engineering

Course No: CSE 4326

Course Title: Microprocessors and Microcontrollers Laboratory

Experiment 2: Wifi Communication and building IoT-based systems Using XAMPP and ESP32

Objectives:

The objective of this lab experiment is to set up an ESP32 board to build IoT-based systems. We shall divide this experiment in the following manners:

i) We will monitor temperature and humidity data using a DHT11 sensor that will be connected to ESP32. Then, the real time data shall be sent to a local server, called “XAMPP” where this data will be stored in a database. Additionally, we will display the data on a local web page.

Students shall perform similar gas data monitoring using this “XAMPP” server and interface led buttons to control using the IoT cloud.

ii) Next, students will monitor the gas sensor data and control an LED using the Arduino Cloud IoT platform. This comprehensive exercise encompasses multiple key steps, starting with the setup of the ESP32 device on the Arduino Cloud IoT platform, which involves device registration and the acquisition of essential credentials. Subsequently, students create a virtual "Thing" on the platform, linking it to the ESP32 and defining variables for data exchange, namely Gas_ppm and LED. Ensuring secure data transmission, students configure network credentials and incorporate the generated secret key.

The lab progresses to programming the ESP32, enabling it to process gas sensor data and manage the LED using digital control logic. Finally, students design a user-friendly dashboard within the XAMPP and Arduino Cloud IoT platform, integrating widgets for real-time sensor data visualization and remote LED control. Final outcomes of this experiment will be-

- Proficient ESP32 setup for gas sensor monitoring.
- Secure IoT data transmission using XAMPP and Arduino Cloud.
- Get a basic understanding of how databases work using PHP.
- How to prepare a simple webpage to show sensor data.
- Effective use of virtual "Thing" for data exchange in Arduino Cloud.
- Competent programming for sensor data processing and LED control.

- User-friendly IoT dashboard design for real-time data visualization.

Components required:

Hardware:

- 1) ESP-32 Development Board
- 2) LED
- 3) DHT11
- 4) MQ-2 Gas Sensor
- 5) Breadboard
- 6) Jumper wires
- 7) 10k ohm Resistor

Software:

XAMPP and Arduino Cloud IoT

Theory:

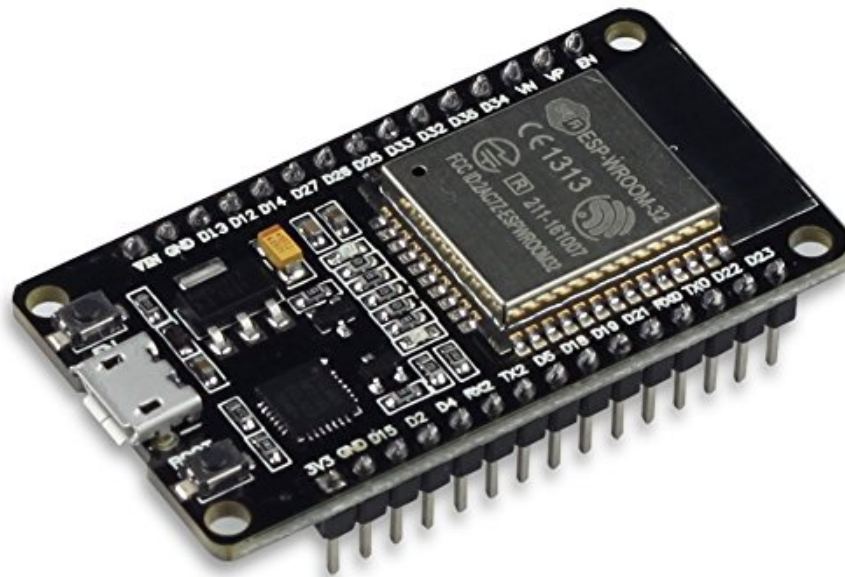


Fig. 1: Diagram of ESP32

The ESP32 is a widely-used System on a Chip (SoC) that plays a key role in developing Internet of Things (IoT) devices. It boasts a range of features that make it a preferred choice for various IoT applications.

- **Built-in Connectivity:** The ESP32 comes equipped with both WiFi and Bluetooth capabilities, making it suitable for IoT devices that need to transmit data over the internet to operate
- **Security Requirements:** The ESP32 offers robust hardware encryption support, which is crucial for ensuring secure communication.
- **Dual Cores:** With two processing cores, the ESP32 enables multitasking and fast parallel processing, making it ideal for handling real-time applications in embedded systems and IoT devices.
- **Storage:** With 1.5 MB of SPIFFS storage, the ESP32 can efficiently store and manage files on external flash memory. This is particularly useful for IoT applications where data storage is a requirement.
- **RAM:** The ESP32 boasts 520 KB of RAM, which is essential for handling data and operations efficiently, especially in IoT devices.
- **Sensor Compatibility:** The ESP32 interfaces seamlessly with a wide range of IoT sensors, supporting UART, I2C, and SPI communication. It features ADC(Analog to digital conversion) and pulse counting for data acquisition and is compatible with capacitive touch sensors, ensuring adaptability to diverse sensor types.

For more information about the ESP32 [click here](#).

Pin Diagram of ESP32:

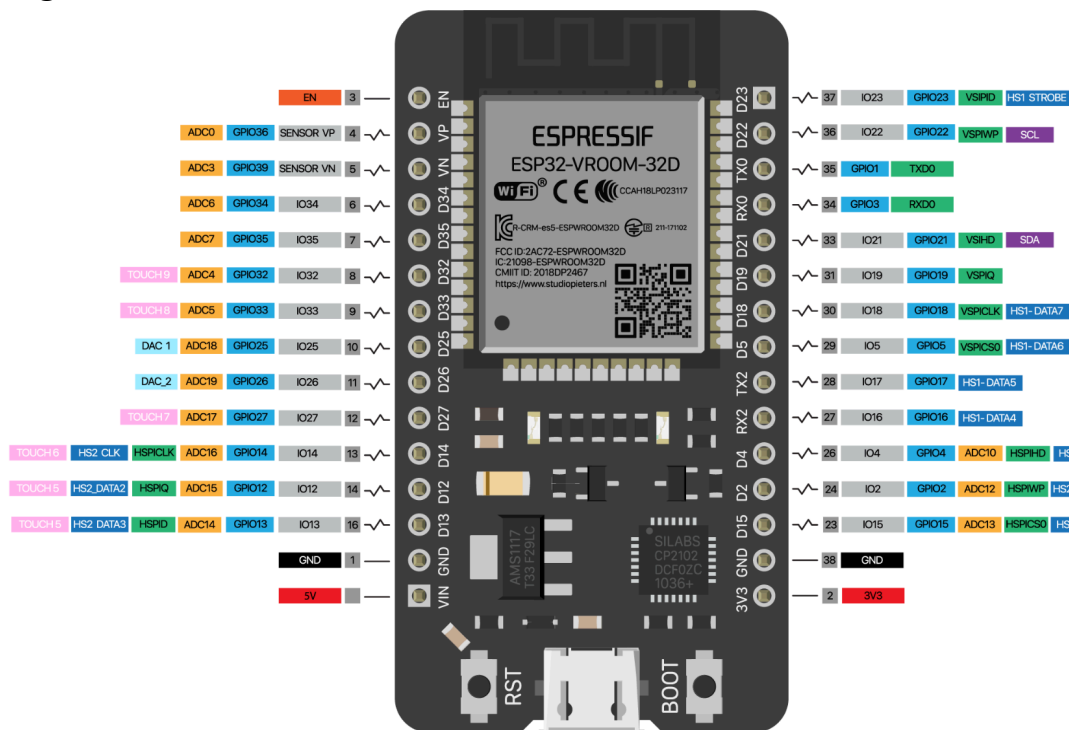


Fig. 2: ESP32 Pin Diagram

The ESP32 peripherals include:

- 18 Analog-to-Digital Converter (ADC) channels
- 3 SPI interfaces
- 3 UART interfaces
- 2 I2C interfaces
- 16 PWM output channels
- 2 Digital-to-Analog Converters (DAC)
- 2 I2S interfaces
- 10 Capacitive sensing GPIO's

Here, the ADC and DAC features are associated with specific static pins. However, UART, I2C, SPI, PWM, etc are assignable independently by code. A more detailed description of each pin is [found here](#).

In this experiment, we're going to explore how to create an IoT device using the versatile ESP32 microcontroller, a DHT11 sensor, an MQ2 gas sensor, and a simple LED. Our goal is to first read temperature and humidity data using the DHT11 sensor and then send the data to be stored on a

database (localhost). After that, we also want to show the variable temperature and sensor data on a local webpage. Then, we shall detect smoke using the MQ2 sensor and seamlessly transmit this data to the local database and Arduino IoT Cloud platform. Moreover, we'll implement a convenient remote control feature via the XAMPP and Arduino IoT cloud dashboard to toggle the LED on and off. This experiment will provide you with valuable insights into the fundamental principles of IoT technology and hands-on experience in building IoT applications.

Installing ESP32 in Arduino IDE:

You can follow these links for installing and troubleshooting in ESP32:

[Installing ESP32 in Arduino IDE \(Windows, Mac OS X, Linux\) | Random Nerd Tutorials](#)
[ESP32 Troubleshooting Guide | Random Nerd Tutorials](#)

Now, let's connect the DHT11 sensor to your ESP32 as shown in Fig. 3.

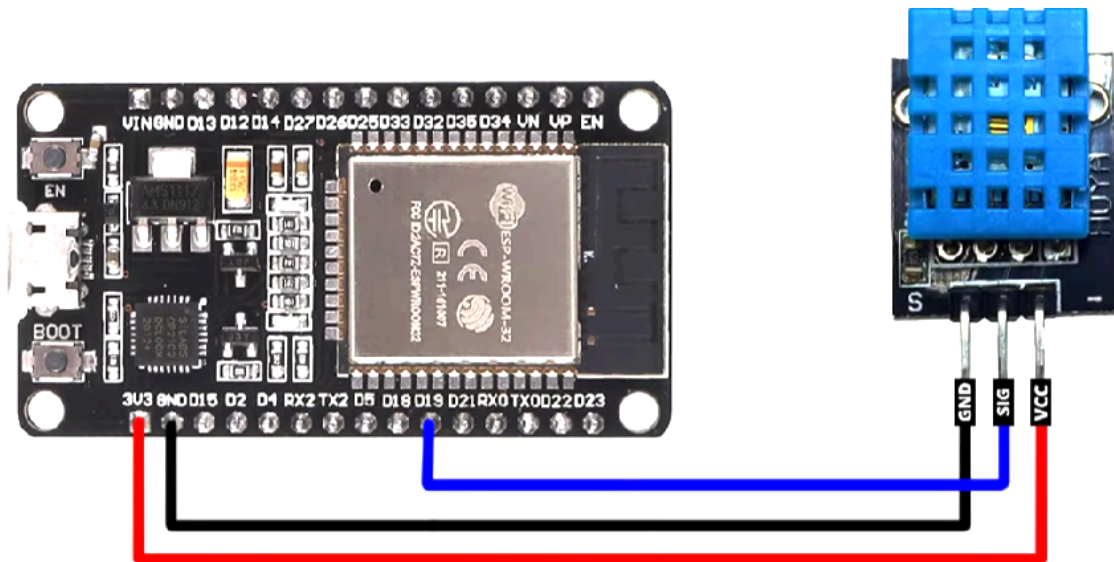
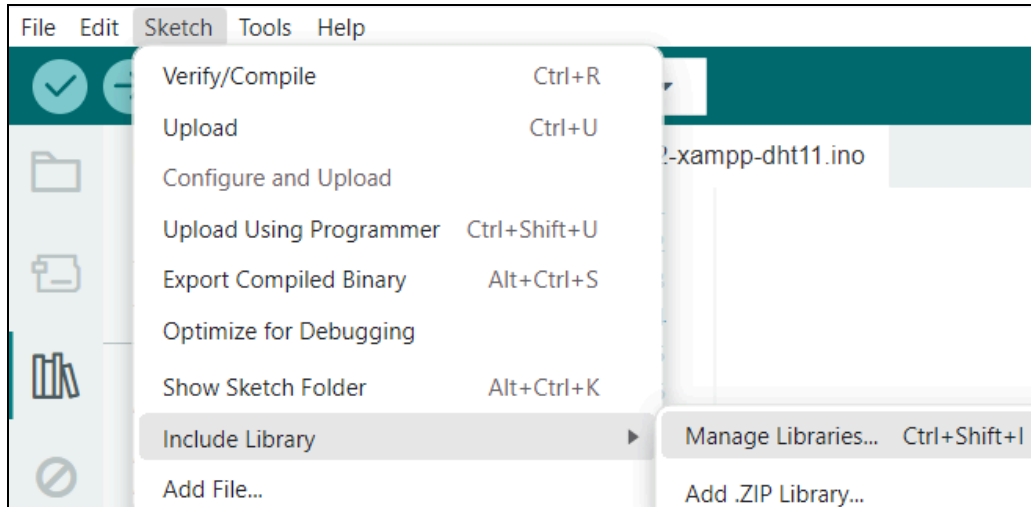
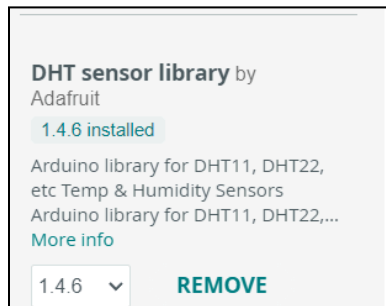


Fig. 3: Connection diagram of DHT11 sensor with ESP32.

Now, we need to install a library for DHT11 in the Arduino IDE to read the temperature and humidity data successfully. Go to Sketch→Include Library→Manage Libraries and then search with the keyword “DHT11” like shown below:



Many libraries will show in the search results. However, install the one given by “Adafruit”. The exact library is shown in the snippet below:



Now, let’s write a simple Arduino code to read the temperature and humidity data from DHT11 and show it to the serial monitor of the Arduino IDE.

Code:

```
#include <DHT.h>
#define DHTPIN 19 //D19
#define DHTTYPE DHT11
DHT dht11(DHTPIN, DHTTYPE); // declare an object “dht11”

int temperature = 0;
int humidity = 0;

void setup() {
  Serial.begin(115200);

  dht11.begin(); // Initialize the sensor
}

void loop() {
```

```
Load_DHT11_Data();  
Serial.println("Temperature: %d °C\n", temperature);  
Serial.println("Humidity: %d %%\n", humidity);  
}  
  
void Load_DHT11_Data() {  
    // _____  
    temperature = dht11.readTemperature(); //Celsius  
    humidity = dht11.readHumidity();  
    // _____  
    // Check if any reads failed.  
    if (isnan(temperature) || isnan(humidity)) {  
        Serial.println("Failed to read from DHT sensor!");  
        temperature = 0;  
        humidity = 0;  
    }  
}
```

Now, save this code, compile and then upload the code to ESP32. It's important to note that sometimes the code uploading to the ESP32 may fail. While this can be due to old USB ports or cable issues, one of the main issues can be due to not rebooting the ESP32. For this, if you see the code is not uploaded to the ESP32, then **you should press and hold the boot button and press the EN button too on the ESP32, located at the bottom right and left corner shown in Fig. 4**, then you click the code upload button on the screen. Wait for the code to upload while holding the boot button.

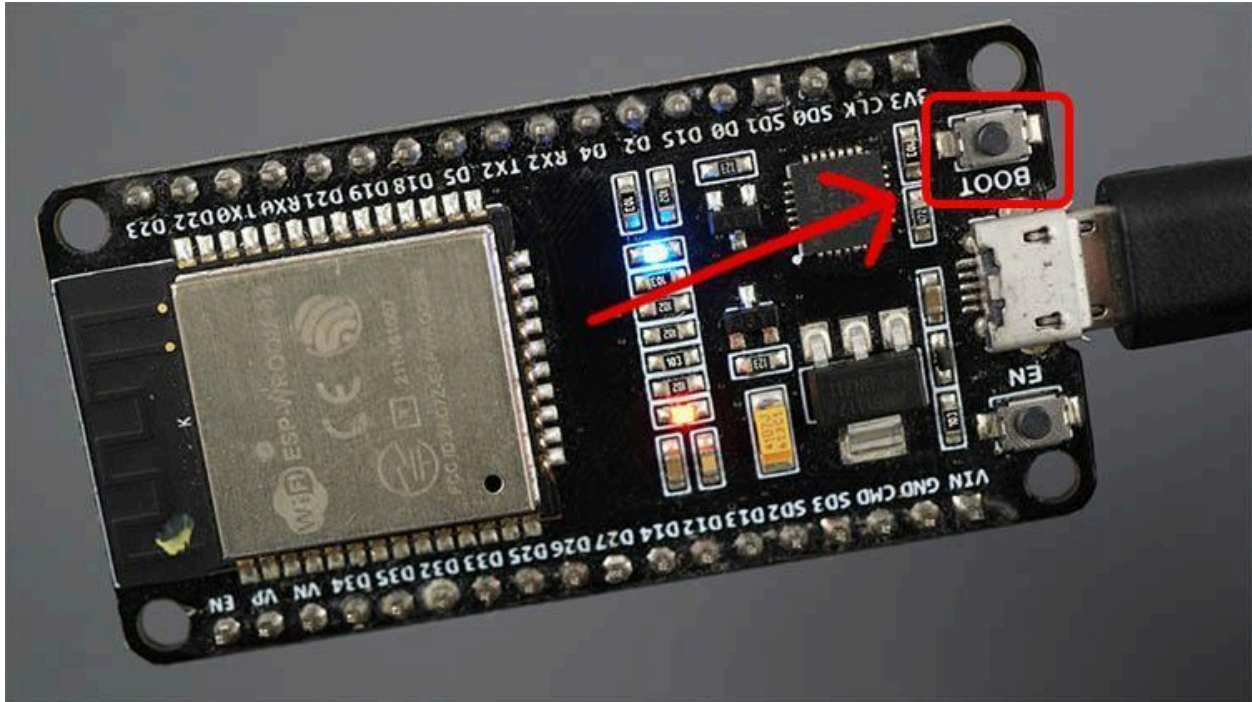


Fig. 4: Boot Button and EN button on ESP32

After uploading to the ESP32, you should be able to see the temperature and humidity data on the **Serial Monitor**.

If you see any kind of problem with uploading issues, first read the error message. You can search the internet with the error messages. Additionally, you can follow the link for troubleshooting ESP32:

[ESP32 Troubleshooting Guide | Random Nerd Tutorials](#)

For more information of DHT11 with Arduino/ESP32, please visit: [Arduino - DHT11 | Arduino Tutorial \(arduinogetstarted.com\)](#)

IoT Cloud Using XAMPP:

Now, we will send the temperature and humidity data from the ESP32 to a local server!

XAMPP is a free and open-source tool that helps developers create and test websites on their own computers before making them live on the internet. It's like a mini-internet server that runs on your computer. The basic components of XAMPP can be seen in Fig. 5. XAMPP includes:

- **Apache:** The main software that serves web pages.
- **MariaDB:** A database that stores website data.
- **PHP and Perl:** Programming languages used to build web applications.
- **XAMPP** is an abbreviation where **X** stands for Cross-Platform, **A** stands for **Apache**, **M**

stands for **M**Y**S**Q**L**, and the **Ps** stand for **P**HP and **P**er**l**, respectively.

XAMPP is easy to install and use, making it perfect for beginners. It allows you to test websites and make sure everything works properly before going live.

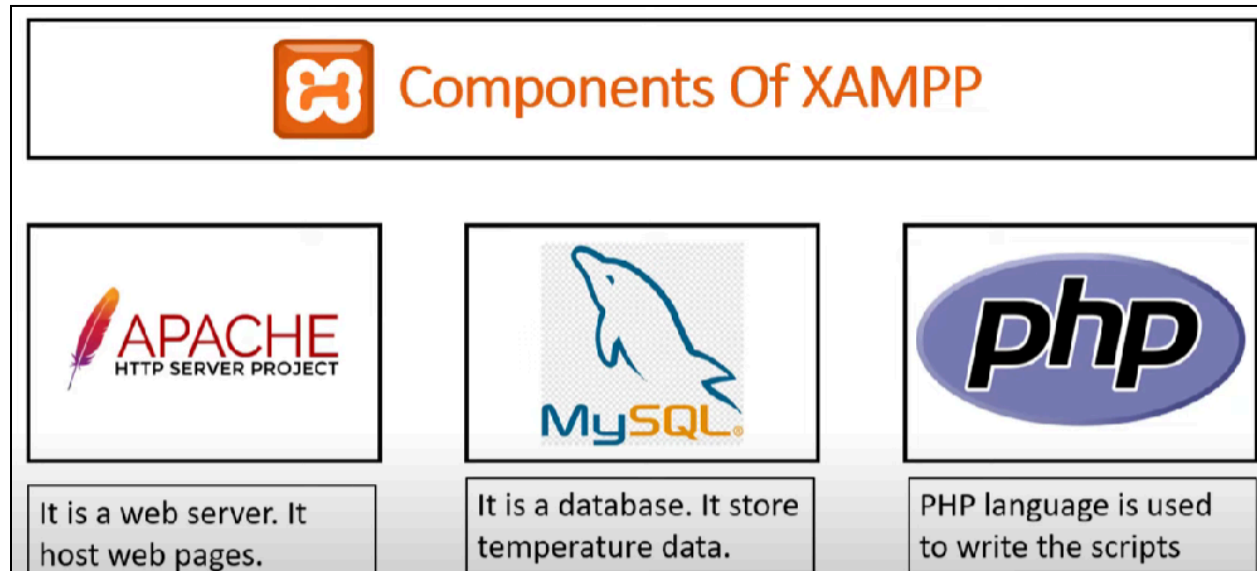


Fig. 5: Basic Components of XAMPP

Apache

What it does: Apache is a web server software. It takes requests from users' web browsers and sends back the requested web pages.

Example: When you type "www.example.com" in your browser, Apache is the software that delivers the website's content to your screen.

MySQL

What it does: MySQL is a database management system. It stores and manages data for websites.

Example: For an online store, MySQL keeps track of product details, customer information, and order history.

PHP

What it does: PHP is a scripting language used for creating dynamic web pages. It runs on the server and generates HTML to send to the user's browser.

Example: When you log into a website, PHP checks your username and password against the data stored in MySQL, and then directs you to your account page.

Interaction in XAMPP among Apache, MySQL & PHP

Example of interaction: Imagine you have a blog. When a visitor comes to your site:

1. **Apache** handles the visitor's request and delivers the web page.
2. The blog post content is stored in **MySQL**.
3. **PHP** retrieves the blog post from MySQL, formats it, and generates the HTML to be displayed.

So, in XAMPP:

- **Apache** serves the web page.
- **MySQL** stores the data.
- **PHP** processes the data and creates the web page content dynamically.

Together, they create a fully functional and interactive website.

Installing XAMPP:

Step-1:

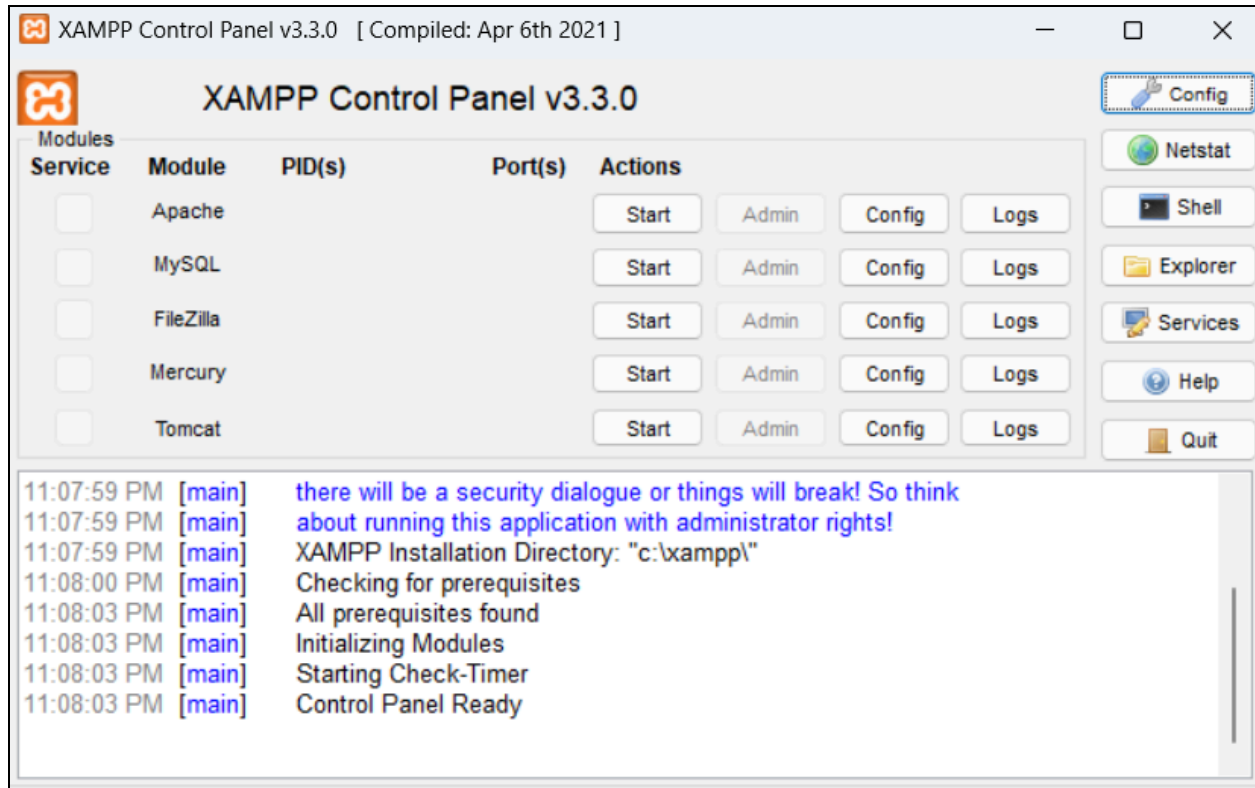
Download the xampp.exe from the website: <https://www.apachefriends.org/> → [XAMPP Installers and Downloads for Apache Friends](#) and install the file. The installation is very straightforward.

Step-2:

To activate the server, we need to go to the installation directory of the XAMPP (for example: “C:\xampp”), then double click the “xampp-control.exe” file.

 xampp_shell.bat	6/24/2024 10:45 PM	Windows Batch File	2 KB
 xampp_start.exe	3/30/2013 6:29 PM	Application	116 KB
 xampp_stop.exe	3/30/2013 6:29 PM	Application	116 KB
 xampp-control.exe	4/6/2021 5:38 PM	Application	3,290 KB

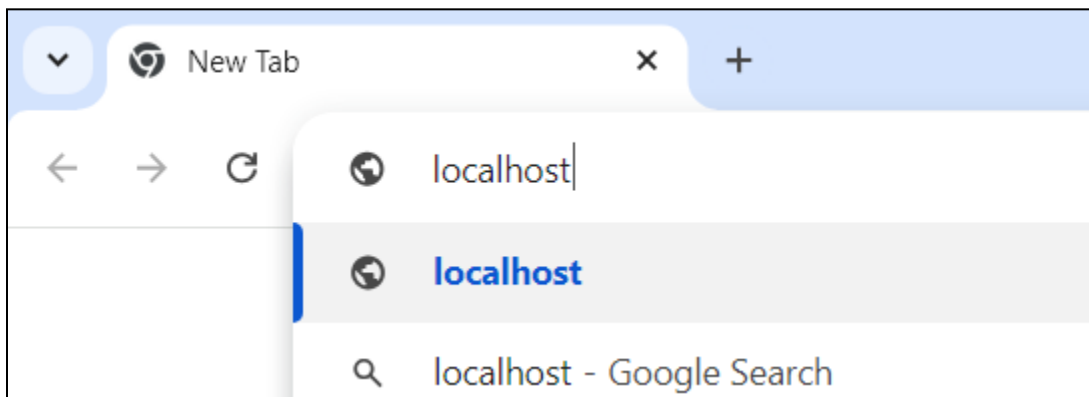
A control panel like the below image will pop up:



Click on the 'start' button in the Actions option to start the Apache and MySQL module. **This is a must to run the XAMPP services while interfacing with ARDUINO or ESP32.**

Step-3:

Let's see if the XAMPP server is working properly! First, type "localhost" in any of your web browser and press enter like the following:



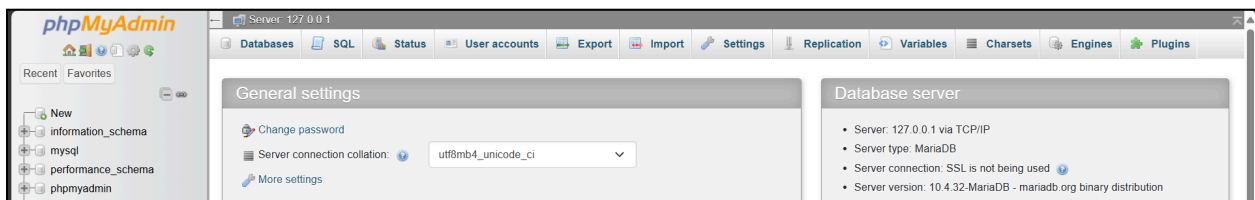
The web server of XAMPP should be displayed like the following:



If you see such a page on your browser that means the “XAMPP” is properly installed.

Step-4:

Click on the ‘phpMyAdmin’ to enter the database. A image like the following will appear:



You can change username and password from the “User accounts” option and a following login screen will appear like shown below **(By default there will be no login screen like below, you have to change in the settings):**



Username: “root”

Password: **there is no default password**

You can change the Username and Password in your personal PC/laptop but do not change for the LAB PCs!

After changing the username and password in your own laptop/pc, you should do the following in the XAMPP installed in your PC. Otherwise, you will not see the login screen!

Open xampp/phpMyAdmin/config.inc and change the line

```
$cfg['Servers'][$i]['auth_type'] = 'config';
```

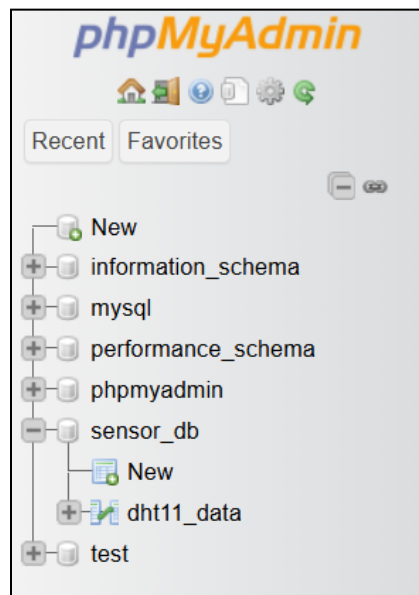
to

```
$cfg['Servers'][$i]['auth_type'] = 'cookie';
```

Creating a Database in XAMPP using “phpMyAdmin”:

Step-1:

Click on the ‘new’ option to create a new database:



Step-2:

Give an appropriate database name in the “Database name” option. Then, click on the “Create” button.

Databases

Create database

Database name: Create

☐ Check all Drop

Database	Collation	Action
☐ information_schema	utf8_general_ci	Check privileges
☐ mysql	utf8mb4_general_ci	Check privileges
☐ performance_schema	utf8_general_ci	Check privileges
☐ phpmyadmin	utf8_bin	Check privileges
☐ sensor_db	utf8mb4_general_ci	Check privileges
☐ test	latin1_swedish_ci	Check privileges

Total: 6

Step-3:

Then give an appropriate database table name as shown in the below image. A single database can have multiple tables. You can set the column value according to your need.

Create new table

Table name:

Number of columns: Create

Here, we will get 2 values from the DHT11 sensor: temperature and humidity. Additionally, we want to maintain a serial number, “ID” and current timestamp, “Datetime” to track the time when we get the data from the sensors. The table should have 4 columns then:

“ID”, “temperature”, “humidity” and “Datetime”. Now, click on “Create” to create the table with 4 columns. An example of the created table is shown below:

Server: 127.0.0.1 » Database: sensor_db » Table: dht11_data

[Browse](#) [Structure](#) [SQL](#) [Search](#) [Insert](#) [Export](#) [Import](#) [Privileges](#) [Operations](#) [Tracking](#) [Triggers](#)

[Table structure](#) [Relation view](#)

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	ID	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐ 2	Temperature	int(11)			No	None			Change Drop More
☐ 3	Humidity	int(11)			No	None			Change Drop More
☐ 4	Datetime	datetime			No	current_timestamp()			Change Drop More

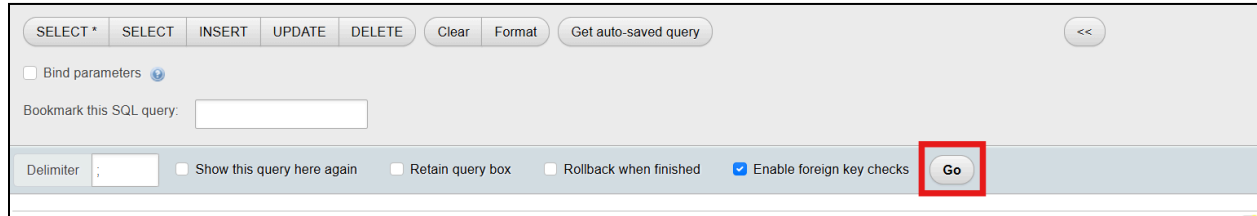
☐ Check all With selected: [Browse](#) [Change](#) [Drop](#) [Primary](#) [Unique](#) [Index](#) [Spatial](#) [Fulltext](#) [Add to central columns](#)

Step-4:

Now, we want to see if our database can be updated properly using SQL command! Go to the SQL option and write the following command:

```
INSERT INTO your_table_name(temperature, humidity) VALUES (10, 20)
```

Then, click on the 'go' command and check if the data table has a new entry with (10, 20).

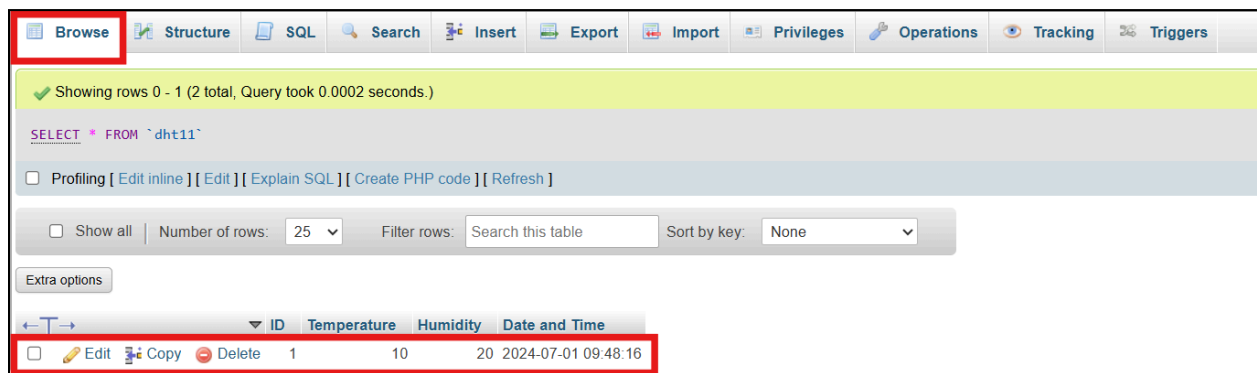


The screenshot shows a SQL query editor with a toolbar at the top containing buttons for SELECT, INSERT, UPDATE, DELETE, Clear, Format, and Get auto-saved query. Below the toolbar, there are checkboxes for 'Bind parameters', 'Bookmark this SQL query', 'Show this query here again', 'Retain query box', 'Rollback when finished', and 'Enable foreign key checks'. The 'Go' button is highlighted with a red box.

If the SQL command executes successfully, a message will be displayed like this:

✓ 1 row inserted. (Query took 0.0054 seconds.)

If the SQL command executes successfully, a message will be displayed like this:



The screenshot shows a database management interface with a toolbar at the top containing buttons for Browse, Structure, SQL, Search, Insert, Export, Import, Privileges, Operations, Tracking, and Triggers. Below the toolbar, there is a message: 'Showing rows 0 - 1 (2 total, Query took 0.0002 seconds.)'. Below this, there is a SQL query: 'SELECT * FROM `dht11`'. Below the query, there are checkboxes for 'Profiling', 'Edit inline', 'Edit', 'Explain SQL', 'Create PHP code', and 'Refresh'. Below these, there are controls for 'Show all', 'Number of rows' (set to 25), 'Filter rows' (Search this table), and 'Sort by key' (None). Below these, there is an 'Extra options' button. Below the 'Extra options' button, there is a table with columns: ID, Temperature, Humidity, and Date and Time. The table contains one row with values: 1, 10, 20, and 2024-07-01 09:48:16. The 'Browse' button is highlighted with a red box.

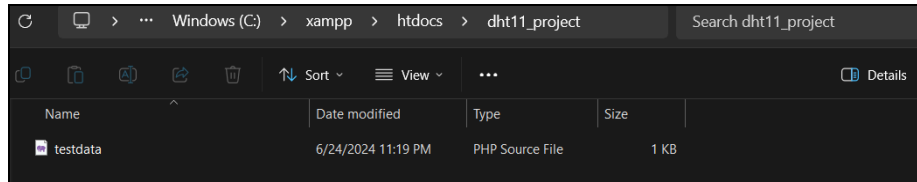
Step-5:

Next step is to write a php script to check whether we can insert fake data into our database through php script to make sure that everything is working correctly before we start sending the real data from the esp32.

To write a PHP script first you need to create a project folder where you will store all the project files. So navigate to the xampp installation directory from C drive here open the "htdocs" folder.

The path: **C:\xampp\htdocs**

This is where you will create your project folder. So create a new folder and give it a name such as dht11 underscore project ("dht11_project"). Next create a new PHP file in this folder and give your file a name for this project. Let's call it "testdata.php".



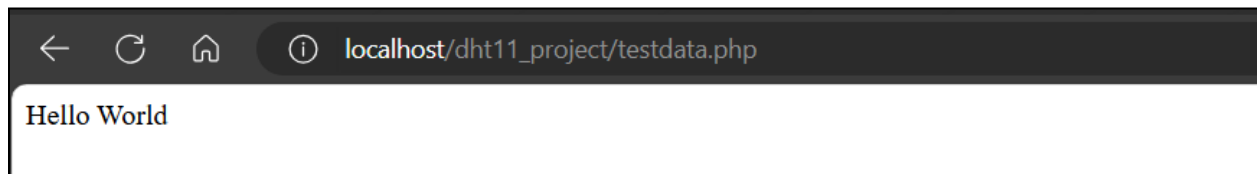
Now open this file with any text editor(vscode/notepad/notepad++ etc.) Then write the following code:

```
<?php  
echo "Hello World<br>";  
?>
```

Here,“<?php ?>” is the opening and closing tags of PHP. And between the tags a message is printed(“echo” is used to print any message in the webpage just like the printf function of C programming language).

Now, save the file by pressing **Ctrl S** and make sure you must save this file in the “dht11_project” folder with PHP extension.

To check whether it is working or not, open your web browser and enter the URL **“localhost\your_project_folder_name\file_name_with_extension”**. For this case it is **“localhost/dht11_project/testdata.php”**. Now press enter to load the page. You'll be able to see the text displayed on the screen.



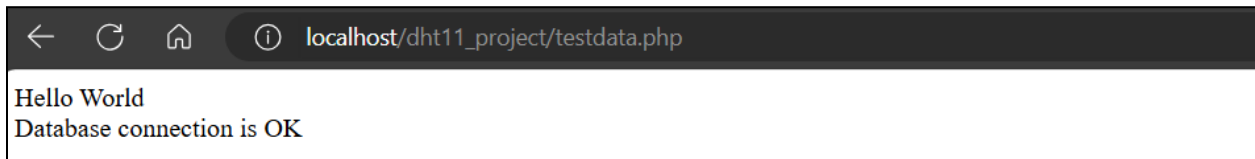
Now let's insert fake data in the database through the PHP script to see if the database is working properly. Update your php file by writing down the following code:

```
<?php  
  
$hostname = "localhost";  
$username = "root";  
$password = "";  
$database = "your database name"; //For this example, it is sensor_db  
  
$conn = mysqli_connect($hostname, $username, $password, $database);  
  
if (!$conn) {  
    die("Connection failed: " . mysqli_connect_error());  
}
```

```
}
```

```
echo "Database connection is OK<br>";  
?>
```

Then save your code and refresh your browser. If everything goes well, the following page will be displayed.



Here, \$hostname, \$username, \$password, and \$database are some variables which are assigned appropriate values to establish the connection with the database. Then, hostname is assigned to localhost, username is root and no password (by default) and database is assigned to the name you have given for your database. For this example, it will be “sensor_db”.

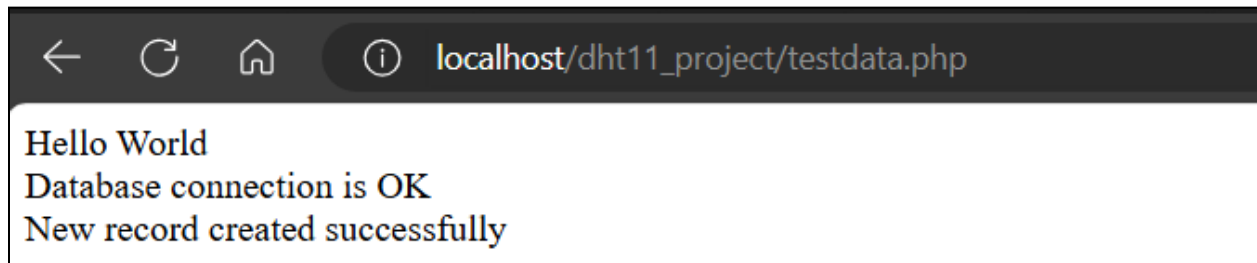
Next to establish a connection to the database, a function is used. The function is “**mysqli_connect**”. This function requires four parameters which are already stored in the variables : hostname, username, password, and database name.

Next an if condition is used to handle the situation if the connection to the database fails. So if the connection fails then the program will stop running and an error message will be displayed to explain the reason for the error. Next if the program successfully connects to the database, then a message will be displayed “database connection is okay”.

Next write the following SQL query to save the data into the table.

```
$sql = "INSERT INTO dht11 (Temperature, Humidity) VALUES (20, 30)";  
if (mysqli_query($conn, $sql)) {  
    echo "\nNew record created successfully";  
} else {  
    echo "Error: " . $sql . "<br>" . mysqli_error($conn);  
}
```

Again save your code and refresh your browser. If everything goes well, the following page will be displayed.



Here, the insertion query which was used before is being stored in the query variable and then executes the SQL query using the “**mysqli_query**” function. Here, a if condition is added which will print out a message to indicate whether the query was successful or not.

To make sure that the data has been inserted into the database go back to phpmyadmin. Then click on sensor_db database. Then click on the dht11 table to select it. You will be able to see the temperature and humidity data that you have just inserted in the table.

				ID	Temperature	Humidity	Date and Time
☐	Edit	Copy	Delete	1	10	20	2024-07-01 09:48:16
☐	Edit	Copy	Delete	2	20	30	2024-07-01 11:54:42

Next one more change in the code will be introduced. Currently the temperature and humidity data is hard-coded in the SQL query which is not a good practice. So instead of hard-coded data, we will use variables. So modify the previous code like this:

```
$t = 1;
$h = 2;

$sql = "INSERT INTO dht11 (Temperature, Humidity) VALUES (". $t . ", ". $h . ")";
if (mysqli_query($conn, $sql)) {
    echo "\nNew record created successfully";
} else {
    echo "Error: " . $sql . "<br>" . mysqli_error($conn);
}
```

Again save your code and refresh your browser. Then go back to the table to check. You will be able to see the temperature and humidity data that you have just inserted in the table through variables.

← T →				ID	Temperature	Humidity	Date and Time
☐	 Edit	 Copy	 Delete	1	10	20	2024-07-01 09:48:16
☐	 Edit	 Copy	 Delete	2	20	30	2024-07-01 11:54:42
☐	 Edit	 Copy	 Delete	3	1	2	2024-07-01 10:33:14

Here, two variables have been declared. The first one is \$t it will be used to store the temperature and the second variable is \$h and it will be used to store the humidity. Then proper changes are made in the sql query to insert the variables instead of hard coded data. This way if you want to change the temperature or humidity values, you can simply modify the variables and the query will automatically use the new values. Using variables is a good programming practice because it makes the code more flexible and easier to modify in the future.

Step-6:

Now, we will write our esp32 code to send data to the XAMPP server. Here first, you need to include libraries that are required for the project to function properly. The first one is the **Wi-Fi library** and this Library will be used for connecting to the Wi-Fi network. The next library is the **HTTP client library** and it will be used to send HTTP requests from ESP32 to our localhost server.

Code:

```
#include <WiFi.h>
#include <HTTPClient.h>

#include <DHT.h>
#define DHTPIN 19 //D19
#define DHTTYPE DHT11
DHT dht11(DHTPIN, DHTTYPE);

String URL = "http://computer_ip/project_folder/project_file.php";

const char* ssid = "ENTER_WIFI_SSID";
const char* password = "ENTER_WIFI_PASSWORD";

int temperature = 0;
int humidity = 0;

void setup() {
  Serial.begin(115200);
```

```

    dht11.begin();

    connectWiFi();
}

void loop() {
    if(WiFi.status()  $\neq$  WL_CONNECTED) {
        connectWiFi();
    }

    Load_DHT11_Data();
    String postData = "temperature=" + String(temperature) + "&humidity=" +
String(humidity);

    HTTPClient http;
    http.begin(URL);
    http.addHeader("Content-Type", "application/x-www-form-urlencoded");

    int httpCode = http.POST(postData);
    String payload = http.getString();

    Serial.print("URL : "); Serial.println(URL);
    Serial.print("Data: "); Serial.println(postData);
    Serial.print("httpCode: "); Serial.println(httpCode);
    Serial.print("payload : "); Serial.println(payload);
    Serial.println("_____");
    delay(5000);
}

void Load_DHT11_Data() {
    //_____
    temperature = dht11.readTemperature(); //Celsius
    humidity = dht11.readHumidity();
    //_____
    // Check if any reads failed.
    if (isnan(temperature) || isnan(humidity)) {
        Serial.println("Failed to read from DHT sensor!");
        temperature = 0;
        humidity = 0;
    }
}

```

```

// _____
Serial.printf("Temperature: %d °C\n", temperature);
Serial.printf("Humidity: %d %%\n", humidity);
}

void connectWiFi() {
  WiFi.mode(WIFI_OFF);
  delay(1000);
  //This line hides the viewing of ESP as wifi hotspot
  WiFi.mode(WIFI_STA);

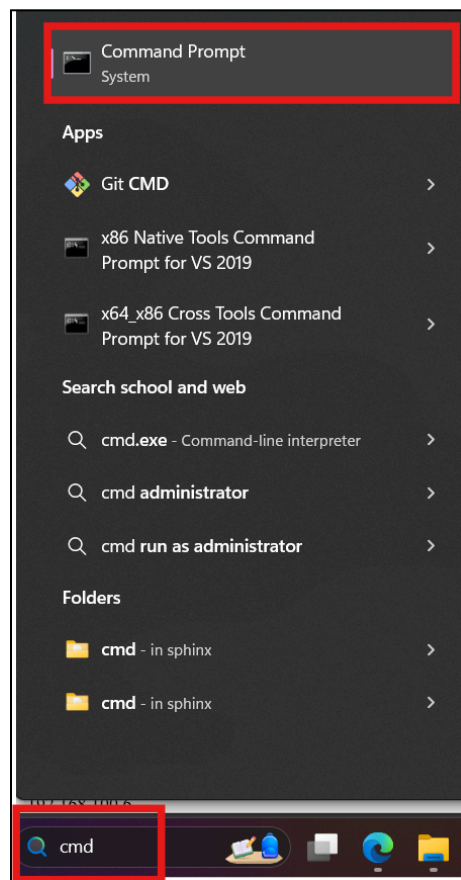
  WiFi.begin(ssid, password);
  Serial.println("Connecting to WiFi");

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.print("connected to : "); Serial.println(ssid);
  Serial.print("IP address: "); Serial.println(WiFi.localIP());
}

```

Next you need to define the address of the web server where you want to send the temperature and humidity data. Here first you need to enter the IP address of your computer which you can easily find by opening the command prompt. So open the **command prompt** (by writing cmd in the search option in windows) on your computer.



Here you can use the “**ipconfig**” command to display the IP address of your computer.

```
C:\Users\HP>ipconfig

Windows IP Configuration

Wireless LAN adapter Local Area Connection* 1:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 2:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Wireless LAN adapter Wi-Fi:

    Connection-specific DNS Suffix  . :
    Link-local IPv6 Address . . . . . : fe80::55a7:56e2:8a7f:4b89%18
    IPv4 Address. . . . . : 192.168.0.111
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.0.1
```

Now here you can see the IP address of the computer in the IPv4 Address part.

Next you just need to enter this IP address in the esp32 code. After that you need to enter the name of the folder where you have saved your project file and then the filename of the php script. For this example, the folder name is **dht11_project** and the filename is **testdata.php**.

Now the URL is ready. **If you are not sure about the URL, you can copy it and then paste it into your web browser. If you see a success message in your browser it means the URL is properly configured.** Just remember you must start the Apache and MySQL services before testing the URL. Next two variables are defined. One is ssid and another one is the password that you will use to connect to the Wi-Fi router or mobile hotspot. You should replace the values inside the quotes with the SSID and Password of your own network.

Next two more variables are defined. One for temperature and another one for humidity which will store the sensor data that will be sent to the Local Host. In this example both variables are initialized to 0.

In the setup function, **Serial.begin()** function is used to set up serial communication between the esp32 and your computer. Next you need to establish a connection to the Wi-Fi router by calling **connectWiFi()** function. This function will attempt to connect to the Wi-Fi network using the SSID and Password that you have defined earlier. This function is added below the loop function. This function contains a sample code that is used in many esp32 projects that require Wi-Fi connectivity. It is a standard code for connecting to any Wi-Fi network. You can also use this function in other projects you make with the esp32 board so you don't have to create a new function every time you want to connect to a Wi-Fi network.

Next let's move in the **loop function**. Here first a checking is done whether the esp32 is connected to the Wi-Fi network. If it is not then it calls the connect Wi-Fi function to attempt to reconnect to the Wi-Fi.

Next a string variable called **postData** is created which holds the temperature and the humidity data that you want to send to the local server. Here first the temperature data is added ("temperature=" + String(temperature)) which is 0. It is the same temperature value that you have defined above. Next the same way the humidity value which is 0 is added ("&humidity=" + String(humidity)). The complete string is look something like this:

temperature=0&humidity=0

Now you have defined the post data string with temperature and humidity data stored in it. Now you will send this data to the server using the HTTP post request. So you will create a new HTTP client object called **http**. It will be used to send an HTTP post request to the web server.

Next let's establish a connection to the xampp server. The **begin function (http.begin(URL))** of the HTTP client object is used to connect the esp32 to the web page which is defined in the URL variable. In this example it is **testdata.php** which you have created earlier. So the esp32 will connect to the testdata.php file and you will send temperature and humidity data to this file. To

send the temperature and humidity data to the server you will use the post method (**http.POST(postData)**). This method will send an HTTP post request to the server with the temperature and humidity data you have stored in the postData variable. An HTTP post method will send it to the testdata.php file. Now the temperature and humidity data is received at the server, so you need to store it to the database table.

Now, you need to modify your code of the testdata.php file like this:

```
if(isset($_POST["temperature"]) && isset($_POST["humidity"])) {  
  
    $t = $_POST["temperature"];  
    $h = $_POST["humidity"];  
  
    $sql = "INSERT INTO dht11 (temperature, humidity) VALUES (". $t .",  
    ". $h .")";  
  
    if (mysqli_query($conn, $sql)) {  
        echo "\nNew record created successfully";  
    } else {  
        echo "Error: " . $sql . "<br>" . mysqli_error($conn);  
    }  
}
```

First a check is done, if there is any valid data received or not using the if condition and if you have some valid temperature and humidity data then you will store that data into these variables (**\$t**, **\$h**). So first you will store the temperature data into the \$t variable that you have defined for temperature. The same way you will store the humidity data to \$h variable. These changes will make the testdata.php file ready to store the temperature and humidity data into the database coming from the esp32.

Next, in the loop function of the arduino ide, a httpcode variable is used to store the response code from the server. If the data is successfully posted to the server then you will receive the response 200. Next another variable called payload is defined. It will store the response message from the server. The message looks something like this:

**Hello World
Database connection is OK
 New record created successfully**

These are the same messages that you have seen earlier while testing the PHP file. In case of error, the payload string contains the error message with the reason for the error. Finally for testing purposes the URL, postData, http code and payload variables are printed to the serial monitor. Here a special message is included called header. Header helps the server to understand the type of information sent to it. Now the code is ready to upload.